

卷积神经网络 CNN

李宏毅《机器学习》2025 · Network Architecture Designed
for Image



基于李宏毅老师课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 56 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=9>

目录

1 从影像分类看架构设计	3
2 图片作为输入：三维 Tensor 与全连接的代价	4
2.1 图片在机器眼中是什么？	4
2.2 全连接网络为什么太贵？	4
3 观察一：重要 Pattern 往往是局部的	5
4 观察二：同样 Pattern 可以出现在不同位置	7
5 从 Filter 的故事看 Convolution	9
6 Pooling：下采样与资讯取舍	12
7 完整 CNN：Convolution、Pooling、Flatten、Fully Connected	14
8 应用例子：为什么 CNN 可以下围棋？	15
9 CNN 的限制：不是天然处理缩放与旋转	18
10 本讲综合：CNN 是为影像写进网络里的先验	19

1 从影像分类看架构设计

这一讲开始讨论 network architecture 的设计。CNN 的重点不只是「有一种叫 convolution 的运算」，而是展示一个更一般的思想：**如果任务本身有结构，我们可以把这些结构写进网络架构里，让模型少学一些不必要的自由度。**

本讲的任务是 image classification：输入一张图片，输出图片中物件所属类别。若类别有 K 种，目标标签通常写成 K 维 one-hot vector；网络输出经过 softmax 后得到分布 $\mathbf{y}'$ ，训练时让 $\mathbf{y}'$ 与真实标签 $\hat{\mathbf{y}}$ 的 cross-entropy 越小越好。



3

图 1: 影像分类任务：输入固定大小图片，输出类别。画面依据：约 01:45–03:10，字幕说明输入图像大小固定、输出用 one-hot label 与 softmax/cross-entropy。

在这堂课里，老师一直把 CNN 放在 P08 的泛化观点下理解：fully connected network 的函数集合很大，表达弹性很高，但参数多、候选函数多，也更容易在有限资料上 overfit。CNN 则把一些影像先验写死在架构里，牺牲一部分通用性，换来更适合影像任务的 inductive bias。

CNN 的核心不是更万能，而是更有偏置

Fully connected layer 可以学习任何像素间的连接关系；CNN 则预设「重要 pattern 通常是局部的」以及「同一个 pattern 可以出现在不同位置」。这些预设如果符合任务，就能减少参数、降低过拟合风险，并让模型更容易学到有用特征；如果任务不符合这些预设，CNN 反而可能不是好选择。

2 图片作为输入：三维 Tensor 与全连接的代价

2.1 图片在机器眼中是什么？

一张彩色图片对人来说是视觉画面，对机器来说是数字阵列。若图片大小为 100×100 ，并且有 RGB 三个 channel，它就是一个

$$100 \times 100 \times 3$$

的三维 tensor。前两个维度是空间位置，高和宽；第三个维度是颜色 channel。每个 pixel 在每个 channel 上都有一个强度数值。

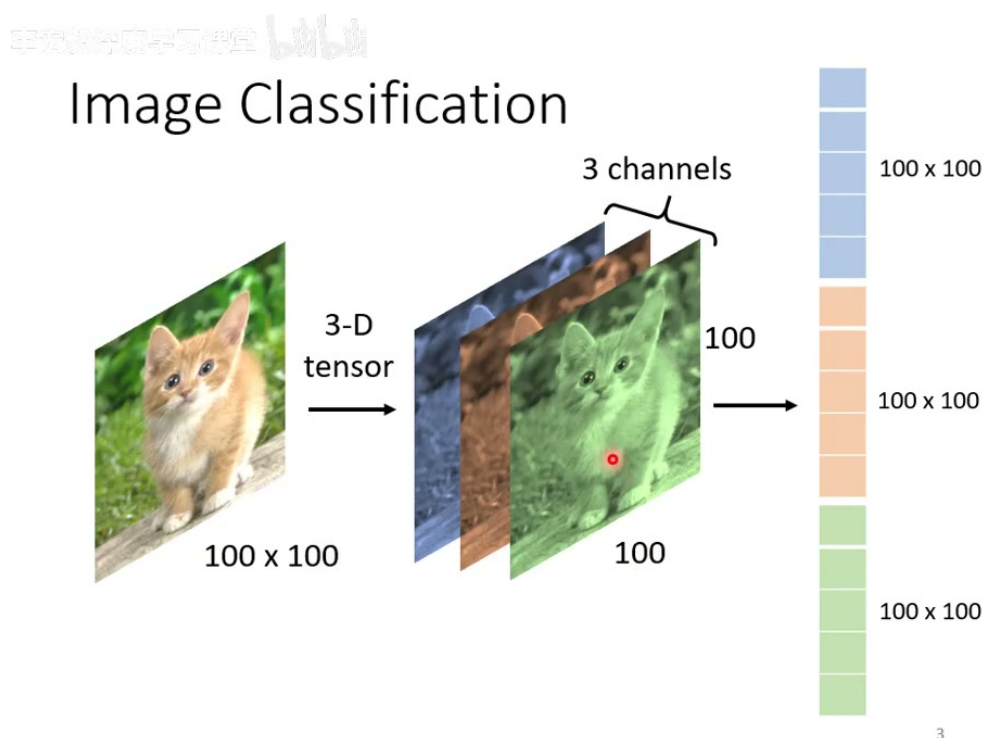


图 2: 彩色图片可看成三维 tensor: 宽、高与 RGB 三个 channel; 若拉直, 则变成长度 $100 \times 100 \times 3$ 的向量。画面依据: 约 03:15–05:40。

到目前为止，课程中最常见的输入是向量。最直觉的做法就是把图片拉直：

$$x \in \mathbb{R}^{100 \times 100 \times 3} = \mathbb{R}^{30000}.$$

这样就可以把图片丢进 fully connected network。这个做法在数学上没有问题，但在参数量上很快变得昂贵。

2.2 全连接网络为什么太贵？

假设第一层有 1000 个 neuron。每个 neuron 都要对输入向量的每一维放一个 weight，因此第一层权重数约为

$$1000 \times 100 \times 100 \times 3 = 3 \times 10^7.$$

这还只是第一层。若图片更大、第一层 neuron 更多，参数量会继续膨胀。

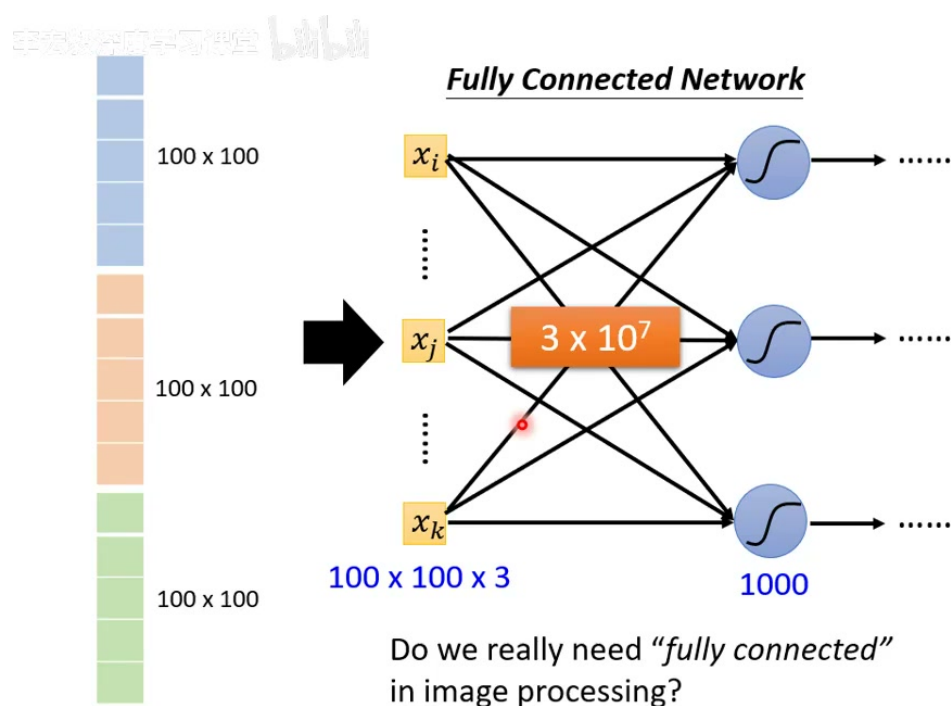


图 3: 把 $100 \times 100 \times 3$ 图片拉直后接到 1000 个 neuron，第一层就有约 3×10^7 个 weight。画面依据：约 05:45–06:35。

参数多并不自动等于不好。参数多通常意味着模型更有弹性，理想 loss 可能更低；但在资料有限时，模型也更容易选择到只适合训练集的函数。CNN 要做的事，就是利用影像任务本身的结构，删除一些不必要的连接关系。

不要把参数减少理解成单纯省内存

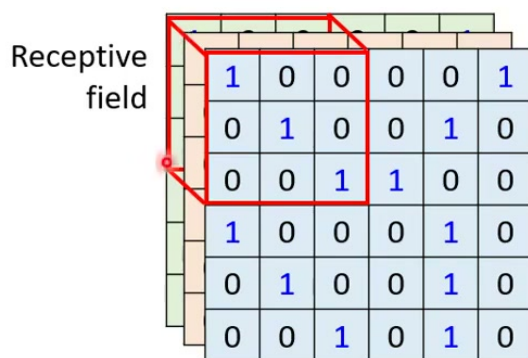
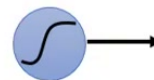
CNN 当然能减少计算和参数，但这不是全部意义。更重要的是它改变了模型家族：它不再允许每个 neuron 任意看全图，也不再允许同一类 pattern 在不同位置各自学一套完全独立参数。这些约束让模型更偏向影像问题中常见的规则。

3 观察一：重要 Pattern 往往是局部的

在判断一张图里有没有鸟时，人并不一定逐像素理解整张图。我们往往看见鸟嘴、眼睛、爪子、羽毛等关键 pattern，就做出判断。机器也可以有类似设计：某些 neuron 负责侦测小 pattern，而小 pattern 不需要看完整张图。

例如要判断某个小区域有没有鸟嘴，通常看局部范围就够了；不需要把整张 100×100 图都输入给这个 neuron。因此 CNN 的第一项简化是 **receptive field**。

Simplification 1



8

图 4: Receptive field: 一个 neuron 只看图片中的局部区域, 而不是整张图。画面依据: 约 10:45–11:55。

若 receptive field 大小为 3×3 , 且覆盖全部 3 个 channel, 则一个 neuron 的输入只有

$$3 \times 3 \times 3 = 27$$

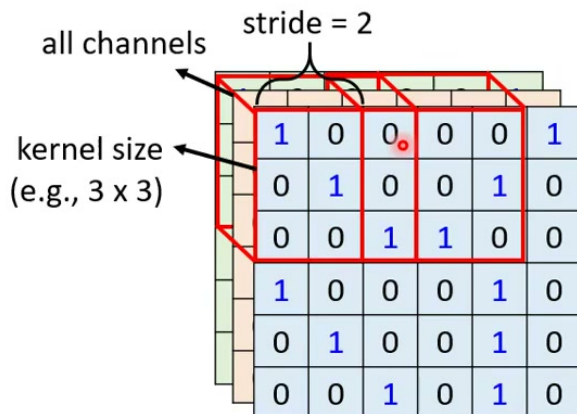
个数值。它只需要 27 个 weight 加一个 bias, 而不是对整张图的 30000 个数值都放权重。

典型设定包括:

- **all channels**: 一般影像 CNN 的 receptive field 会覆盖所有输入 channel。
- **kernel size**: 只描述空间大小, 如 3×3 、 5×5 。
- **stride**: receptive field 在图片上移动的步长, 常见为 1 或 2。
- **padding**: 当 receptive field 超出边界时补值, 最常见是 zero padding。
- **multiple neurons per field**: 同一个局部区域通常由一组 neuron 处理, 如 64 个。

Simplification 1 – Typical Setting

Each receptive field has a set of neurons (e.g., 64 neurons).



10

图 5: 典型 receptive field 设置: 覆盖所有 channel, kernel size 如 3×3 , stride 决定移动间隔, 每个 receptive field 可有一组 neuron。画面依据: 约 15:45–18:10。

为什么小 kernel 也能看见大 pattern ?

单层 3×3 filter 只看局部, 但多层 convolution 堆叠后, 后一层的一个位置对应到原图中更大的范围。例如第二层的 3×3 区域, 其每个值已经来自第一层的局部卷积结果, 因此映回原图时可能对应 5×5 或更大区域。网络越深, 有效 receptive field 越大。

4 观察二: 同样 Pattern 可以出现在不同位置

第二个影像先验是: 同一个 pattern 可以出现在图片的不同区域。鸟嘴可能在左上角, 也可能在中间; 但「侦测鸟嘴」这件事本身不应该因为位置不同就完全重学一套参数。

若每个位置都有独立 neuron 去学「鸟嘴侦测器」, 参数会大量重复。CNN 因此引入 **parameter sharing**: 不同位置的 neuron 可以共享同一组 weight, 只是它们看的输入区域不同。

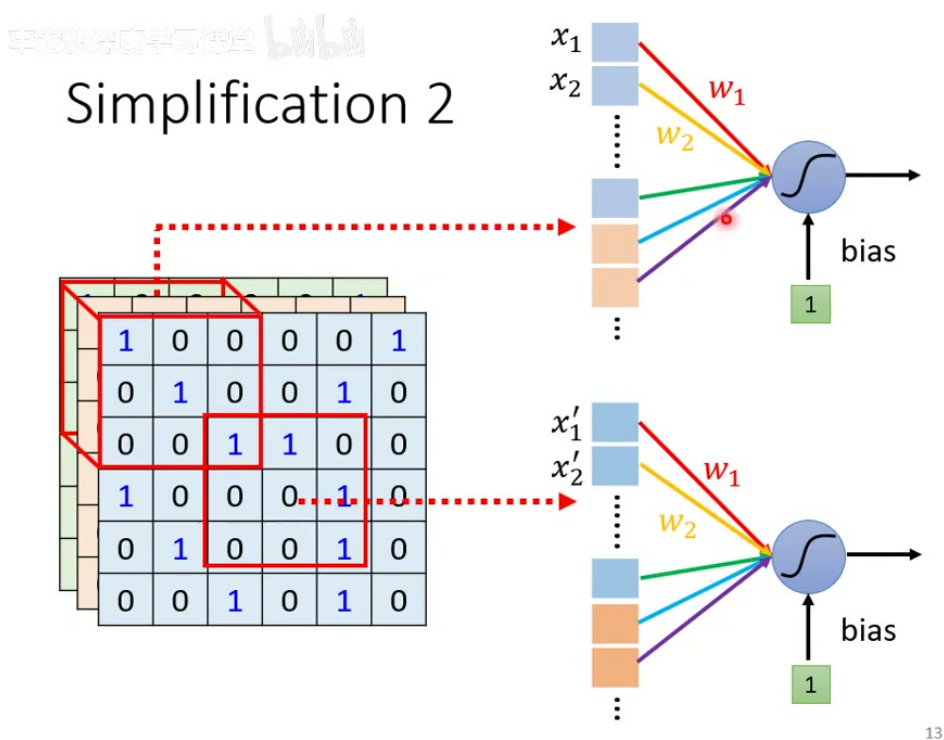


图 6: Parameter sharing: 不同 receptive field 中的 neuron 可使用同一组 weight; 参数相同但输入区域不同, 因此输出不必相同。画面依据: 约 22:45–24:30。

用公式写, 如果两个位置的局部输入分别是 $\mathbf{x}$ 与 $\mathbf{x}'$, 共享参数为 $\mathbf{w}$, 则输出可写成

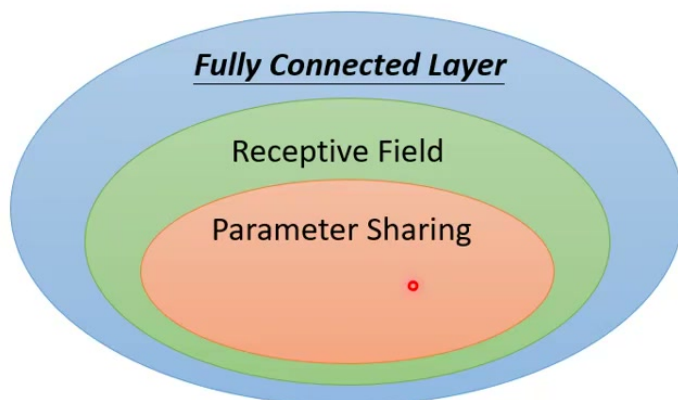
$$a = \sigma(\mathbf{w}^T \mathbf{x} + b), \quad a' = \sigma(\mathbf{w}^T \mathbf{x}' + b).$$

虽然 $\mathbf{w}$ 和 b 相同, 只要 $\mathbf{x} \neq \mathbf{x}'$, 输出通常也不同。共享参数不是让所有位置输出一样, 而是让同一种侦测器在所有位置复用。

Convolutional layer = receptive field + parameter sharing

Receptive field 限制每个 neuron 只看局部; parameter sharing 限制不同位置的同类 neuron 共用参数。两者合起来, 就是 convolutional layer 的核心。它比 fully connected layer 更受限制, model bias 更大, 但这个 bias 正是为影像设计的。

Benefit of Convolutional Layer



- Some patterns are much smaller than the whole image.
- The same patterns appear in different regions.

16

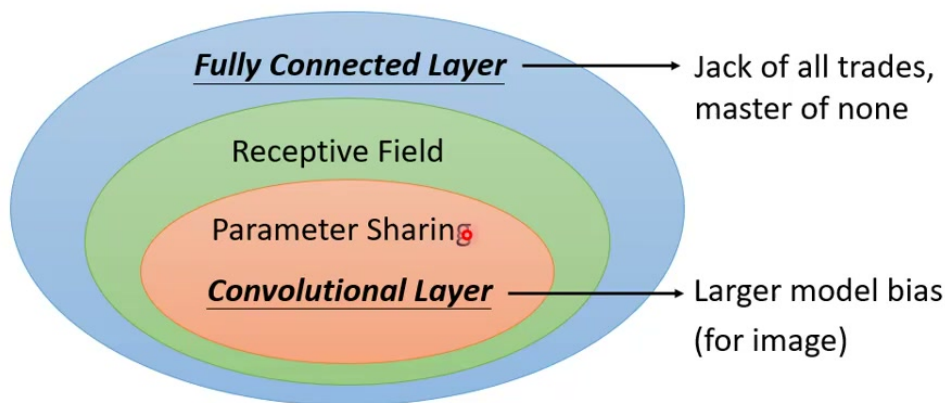
图 7: Convolutional layer 是 fully connected layer 的受限版本：先加入 receptive field，再加入 parameter sharing。画面依据：约 26:20–28:55。

这也解释了为什么 CNN 不是“比较强的全连接层”。它从表达能力上其实更窄，但窄在正确方向上：它把影像任务里不太需要的自由度拿掉。

5 从 Filter 的故事看 Convolution

同一件事也可以用 filter 来讲。一个 convolutional layer 里有一组 filters。每个 filter 是一个小 tensor，例如 $3 \times 3 \times C$ ，其中 C 是输入 channel 数。Filter 的参数要通过训练学出来；训练完成后，一个 filter 可以理解为某类局部 pattern 的侦测器。

Benefit of Convolutional Layer



- Some patterns are much smaller than the whole image.
- The same patterns appear in different regions.

16

图 8: Filter 版本的 CNN 叙事: 每个 filter 是 $3 \times 3 \times channel$ 的小 tensor, 用来在图片中侦测某种局部 pattern。画面依据: 约 29:40–30:35。

卷积运算可以理解成: 把 filter 放到图片某个位置, 取局部 patch 与 filter 的 inner product, 再把 filter 依照 stride 滑到下一个位置重复计算。

若输入 patch 为 $X_{i:i+2,j:j+2}$, filter 为 $W \in \mathbb{R}^{3 \times 3}$, 则单个输出可写成

$$z_{i,j} = \sum_{u=0}^2 \sum_{v=0}^2 W_{u,v} X_{i+u,j+v} + b.$$

彩色图片或多 channel feature map 时, 还要再对 channel 维度求和。

Convolutional Layer

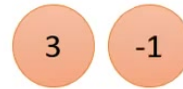
stride=1

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



19

图 9: 卷积的数值例子: filter 与局部 patch 做 inner product; filter 扫过整张图后得到一张输出图。画面依据: 约 31:30-33:20。

一个 filter 扫完整张图会得到一张二维输出; 多个 filter 会得到多张输出。这些输出合在一起叫 **feature map**。若一层有 64 个 filters, 输出就有 64 个 channel。下一层 convolution 的 filter 深度必须等于前一层输出 channel 数。

例如第一层输入是 RGB 图片, filter 形状可能是 $3 \times 3 \times 3$; 若第一层有 64 个 filters, 输出 feature map 有 64 个 channel; 第二层 filter 就可能是 $3 \times 3 \times 64$ 。

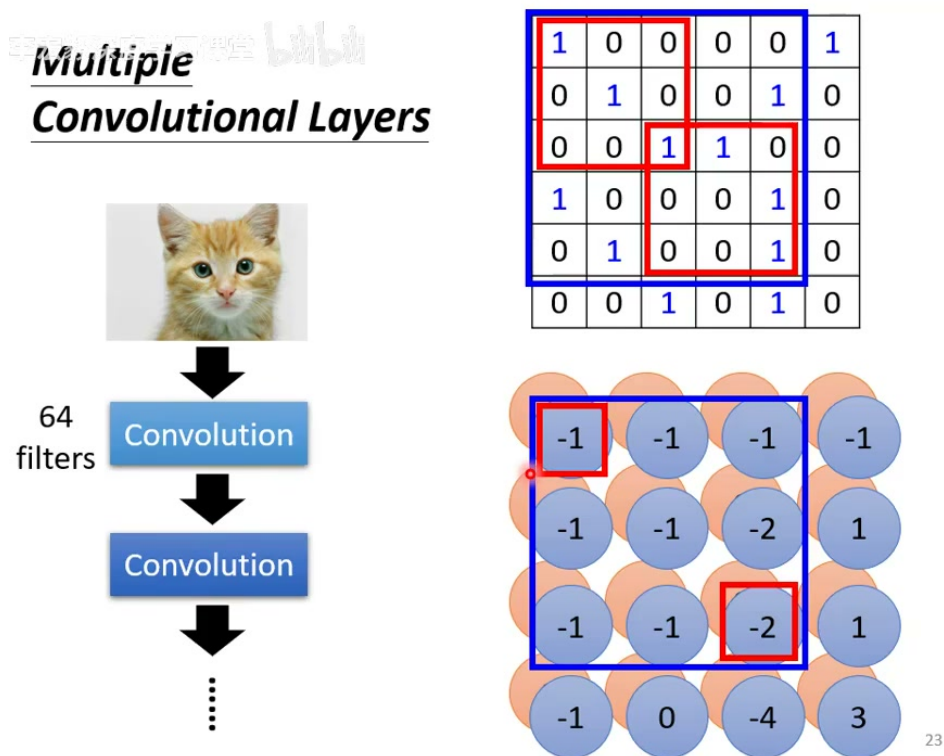


图 10: Neuron 版本与 filter 版本是同一件事：共享参数的 neuron，对应到一个 filter 扫过整张图。画面依据：约 37:40–39:25。

为什么叫 convolution ?

从 neuron 视角看，是多个不同位置的 neuron 共用同一组参数；从 filter 视角看，是同一个 filter 在整张图片上滑动计算。后一种操作就是 convolutional layer 名字里的 convolution。

6 Pooling：下采样与资讯取舍

除了 convolution，经典 CNN 常搭配 pooling。Pooling 来自第三个观察：把图片做适度 sub-sampling，通常不会改变图片里的物件。例如一张鸟的图片缩小后还是鸟。

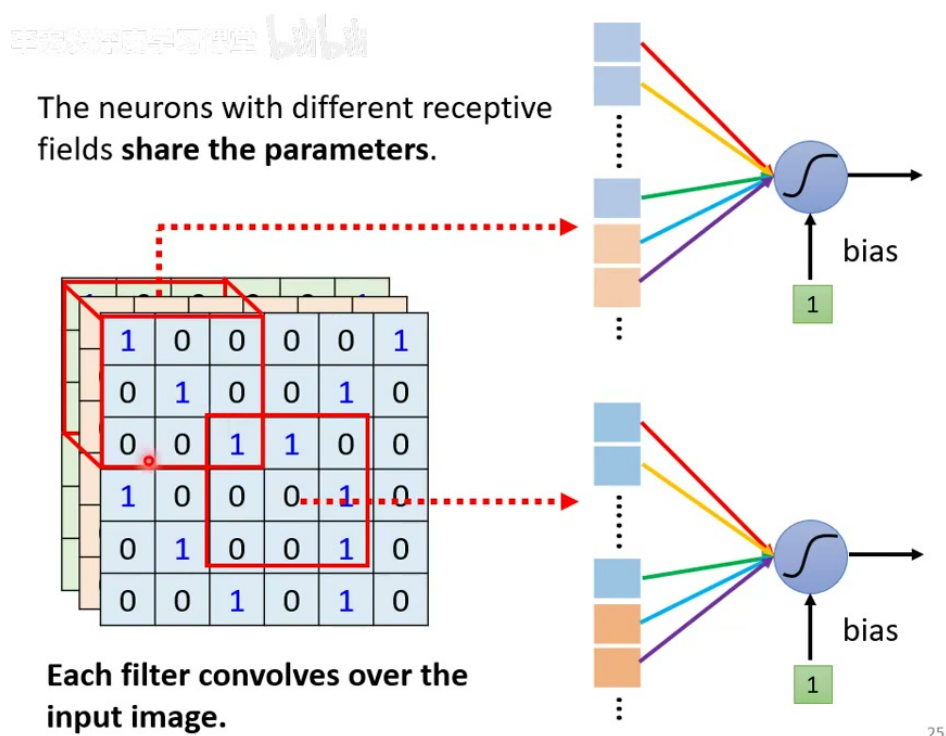


图 11: Pooling 的直觉来源：适度缩小图片不改变物件类别。画面依据：约 40:10–40:35。

Pooling 本身没有可学习参数，更像一个固定 operator。以 max pooling 为例，把 feature map 划成若干小块，如 2×2 ，每块取最大值作为代表：

$$y_{i,j} = \max_{(u,v) \in \Omega_{i,j}} x_{u,v}.$$

这样空间大小变小，channel 数不变。例如 $4 \times 4 \times 64$ 经过 2×2 max pooling 后可能变成 $2 \times 2 \times 64$ 。

Pooling – Max Pooling



28

图 12: Convolution 后得到多 channel feature map; pooling 会降低空间解析度, 但通常保留 channel 数。画面依据: 约 41:10–42:45。

Pooling 不是永远该用

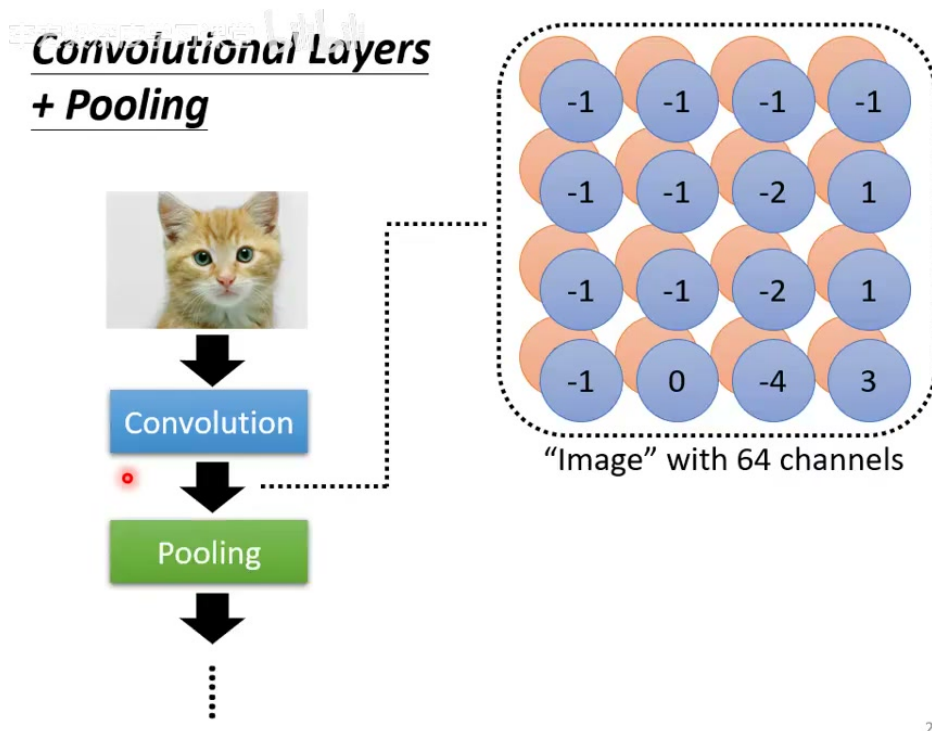
Pooling 会减少计算量, 也能带来一定程度的位置稳定性; 但它也丢掉资讯。若任务非常依赖精细空间位置, 例如围棋, 随便下采样可能改变问题本身。近年的很多视觉架构也会减少或取消 pooling, 改用更深的 convolution 或其他下采样设计。

7 完整 CNN: Convolution、Pooling、Flatten、Fully Connected

经典 CNN 的整体结构通常是:

Image $\rightarrow$ Convolution $\rightarrow$ Pooling $\rightarrow \dots \rightarrow$ Flatten $\rightarrow$ Fully Connected $\rightarrow$ Softmax.

Convolution/Pooling 负责把原始像素转成越来越抽象的 feature map; Flatten 把矩阵状 feature map 拉直成向量; fully connected layers 再根据这些高层特征做最终分类。



29

图 13: 完整 CNN: 多次 convolution/pooling 后 flatten, 再接 fully connected layers 和 softmax 输出类别。画面依据: 约 43:55–44:45。

这里可以把前面的三种设计合起来看:

- Locality: filter 只看局部, 减少单个探测器输入维度。
- Parameter sharing: 同一个 filter 扫过全图, 减少重复学习。
- Hierarchy: 多层 convolution 让低层 pattern 组合成高层 pattern。

8 应用例子: 为什么 CNN 可以下围棋?

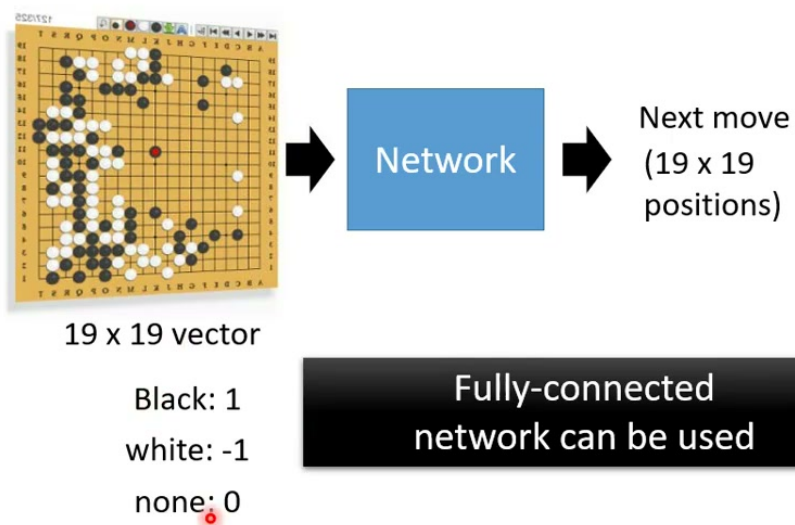
CNN 不只用于自然图片, 也曾用于 AlphaGo 的围棋模型。围棋可以写成分类任务: 输入棋盘状态, 输出下一步落子位置。棋盘有 19×19 个位置, 因此下一步位置可看成 19×19 个类别之一。

棋盘也可以看成一张小图片: 空间解析度是 19×19 , 每个位置的 channel 描述该点状态, 例如黑子、白子、空位、是否被叫吃等。在 AlphaGo 的论文描述中, 输入是

$$19 \times 19 \times 48$$

的 image stack。

Application: Playing Go



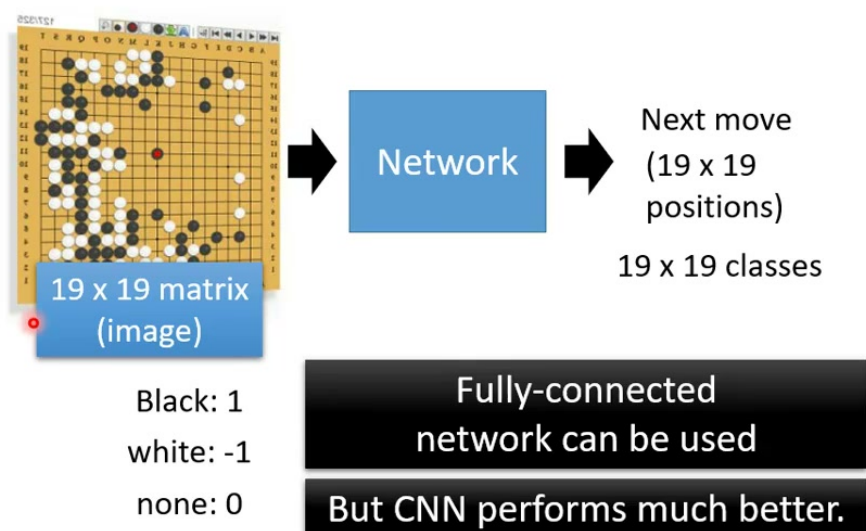
31

图 14: 围棋可转成影像式输入: 19×19 棋盘位置加上多组 feature planes, 输出是 19×19 个可能落子位置之一。画面依据: 约 45:05–48:15。

为什么围棋适合 CNN? 因为它和影像有两个相似结构:

- 局部 pattern 重要: 某些棋形只需要看小范围就能判断, 例如叫吃。
- 同样 pattern 可在不同位置出现: 同一种棋形可能出现在棋盘任何角落。

Application: Playing Go



31

图 15: 围棋与影像共享 CNN 的两项先验: 重要 pattern 通常局部, 同样 pattern 可出现在不同区域。画面依据: 约 48:20–49:45。

但围棋不适合 pooling。自然图片缩小一点仍是同一物件, 棋盘却不能随便删掉某些行列; 一个位置消失, 棋局就变了。老师特意查 AlphaGo 的论文附录, 指出其 policy network 使用多层 convolution、zero padding、ReLU、softmax, 但没有 pooling。

Why CNN for Go playing?

- Subsampling the pixels will not change the object



Pooling

How to explain this???

Neural network architecture. The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Table 3 additionally show the results of training with $k = 128, 256$ and 384 filters.

33

图 16: AlphaGo 网络结构摘录: 输入 $19 \times 19 \times 48$, 第一层 5×5 filters, 后续 3×3 filters; 关键点是没有 pooling。画面依据: 约 51:10–52:35。

架构的应用之道存乎一心

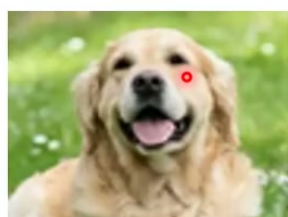
看到影像 CNN 常用 pooling, 不代表所有 CNN 任务都该 pooling。每个架构元件都对应某种假设: locality、translation sharing、subsampling invariance。要把 CNN 用到语音、文字或围棋, 必须重新检查这些假设是否符合任务。

9 CNN 的限制: 不是天然处理缩放与旋转

CNN 常被误解成天然具有各种视觉不变性。实际上, 标准 CNN 主要内建的是局部性和位置共享, 并不自动处理缩放、旋转。对人眼来说, 同一只狗放大、缩小、旋转后仍是同一物件; 但对网络输入向量来说, 像素排列已经大幅改变。

To learn more ...

- CNN is not invariant to scaling and rotation (we need data augmentation 😊).



Spatial Transformer Layer



<https://youtu.be/SoCywZ1hZak>
(in Mandarin)

35

图 17: CNN 不天然 invariant to scaling and rotation; 常需 data augmentation 或额外结构补足。画面依据: 约 53:50–55:35。

这解释了为什么实际影像训练常做 data augmentation: 随机裁切、缩放、旋转、翻转、颜色扰动等。Augmentation 的作用不是让资料变多这么简单, 而是让模型在训练中看过同一物件的多种变形, 从而学习到想要的不变性。

10 本讲综合: CNN 是为影像写进网络里的先验

这一讲的主线可以压缩成一句话: CNN 把影像任务中合理的结构假设写进神经网络架构。

- 图片是三维 tensor, 直接拉直接 fully connected layer 会产生大量参数。
- Receptive field 利用局部性: 侦测小 pattern 不必看全图。
- Parameter sharing 利用平移共享: 同一种 pattern 可在不同位置复用同一组参数。
- Filter 扫过图片的过程就是 convolution; 多个 filters 产生多 channel feature map。
- Pooling 利用下采样不改变物件的直觉, 但它不是所有任务都适合。
- CNN 的有效性来自任务先验与架构偏置匹配, 而不是来自“卷积”这个名字本身。

和 P08/P10 的泛化观点连起来

P08 说模型复杂度越大，有限训练集越容易产生训练/真实落差。CNN 的做法不是盲目扩大模型，而是在影像问题上减少不必要的候选函数：不让每个 neuron 任意看全图，不让不同位置重复学习相同侦测器。只要这些限制符合任务，就能同时保留足够表达力，并降低过拟合风险。